

SystemC Virtual Platform — Customer Guide

2026-06-08

Contents

1. Overview	2
2. Simulation Flows	2
2.1 Synopsys Virtualizer (VDK)	2
Prerequisites	2
Reference Build Versions (RISC-V Reference Platform)	2
Environment Setup	3
Building	3
Running	3
Key Files in a VDK Project	3
2.2 Accellera SystemC (Open-Source)	4
Prerequisites	4
Environment Setup	4
Building and Running	4
Key Files in an Accellera Project	4
3. Interface Standards	5
Socket Types	5
Sideband Signals	5
Transport Interface	5
4. Temporal Decoupling and the Quantum	6
4.1 What Is Temporal Decoupling?	6
4.2 The Quantum Keeper	6
4.3 Setting the Quantum in Virtualizer (VDK)	6
4.4 Setting the Quantum in Accellera SystemC	7
4.5 Best Practices for Linux Boot	7
4.6 Memory and Transaction Ordering in Loosely-Timed Simulation	8
5. How to Integrate a New IP	10
5.1 Common Requirements	10
5.2 VDK Flow — Declarative Integration	11
5.3 Accellera Flow — Programmatic Integration	12
6. TLM-2.0 Quick Reference	13
7. Integration Checklist	13

A high-level guide to building and running SystemC-based virtual platforms and integrating new IP models. This document covers two simulation flows and the interface standards that every IP must follow to plug into the platform.

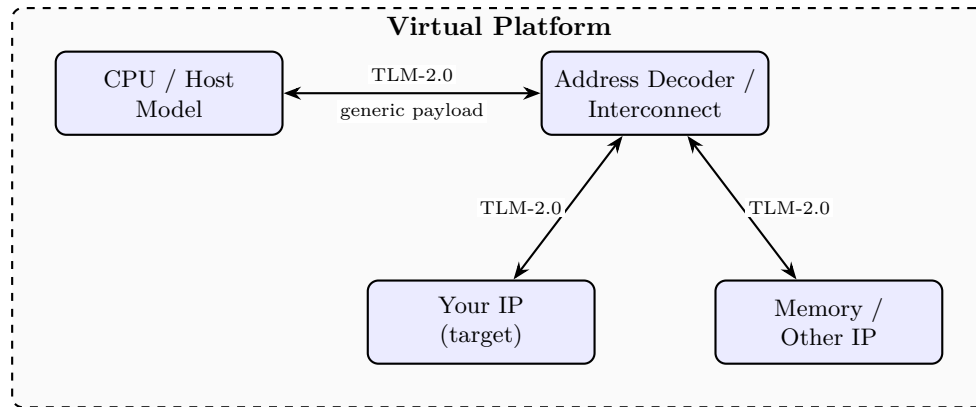
1. Overview

The platform uses IEEE 1666 SystemC and TLM-2.0 as its foundation. Every IP model — whether it is a PCIe controller, an IOMMU, a DMA engine, or a custom accelerator — connects to the rest of the system through the same set of standard interfaces.

Two simulation environments are supported. You choose one based on your project needs:

	Synopsys Virtualizer (VDK)	Accellera SystemC
License	Commercial (Virtualizer Elite)	Open-source / free
Best for	Full-system prototypes with CPUs, buses, and pre-built DesignWare IPs	Standalone IP-level models and unit-test environments
Build system	Virtualizer Studio (GUI / Python API)	CMake + standard C++ toolchain
IP wiring	Declarative JSON (<code>.vdksys</code> system descriptor)	Programmatic C++ (socket binding in <code>sc_module</code>)
Typical entry point	<code>.vpproject</code> opened in Virtualizer Studio	<code>sc_main()</code> in a C++ executable

Regardless of which flow you use, the IP-facing interface contract is identical: TLM-2.0 sockets with `tlm_generic_payload`, plus SystemC signals for clock, reset, and interrupts.



2. Simulation Flows

2.1 Synopsys Virtualizer (VDK)

Synopsys Virtualizer is a commercial SystemC simulation environment designed for building full-system virtual prototypes. It provides a graphical IDE (Virtualizer Studio), a rich library of pre-built CPU and peripheral models (DesignWare, Imperas RISC-V, etc.), and a JSON-based system descriptor format for wiring IPs together.

Prerequisites

Requirement	Details
Synopsys Virtualizer	V-2024.03 or later (Virtualizer Elite license)
Cross-compiler	Matching the target CPU architecture (e.g. <code>riscv64-unknown-linux-gnu-*</code>)
Git LFS	If the project uses LFS for packaged IP archives

Reference Build Versions (RISC-V Reference Platform)

The *Prerequisites* table above lists requirements generically (“Virtualizer V-2024.03 or later”, “a cross-compiler matching the target CPU”). The tables below show a **representative set of versions a RISC-V based reference platform can use to build and boot**, so you can reproduce a known-good configuration. The host/SystemC values come from the Virtualizer environment setup; the RISC-V target values were read directly from the `Linux version` banners embedded in the produced binaries (`vmlinux` and `fw_payload.elf`).

Host / SystemC build environment

Component	Version (reference build)
Host OS distribution	Red Hat Enterprise Linux 8.10 (Ootpa)
Host OS kernel	4.18.0-553.el8_10.x86_64
Host architecture	x86_64
Synopsys Virtualizer	V-2024.03 (Virtualizer Elite)
SystemC build compiler (host)	GCC 9.5 (Synopsys-bundled gcc-9.5-64)
Bundled Python	3.10 (libpython3.10)
SystemC / TLM standard	IEEE Std 1666-2023, TLM-2.0

RISC-V target software stack (guest)

Component	Version (reference build)
Linux kernel	6.6.132 (6.6.y LTS)
Kernel & OpenSBI compiler	riscv64-unknown-linux-gnu-gcc (GCC) 15.2.0 — ABI lp64d
Userspace / BusyBox compiler	riscv64-unknown-linux-musl-gcc (GCC) 15.2.0 — ABI lp64 (soft-float)
Binutils / GNU ld	2.46
Firmware	OpenSBI (built from source) — M-mode fw_payload.elf

Environment Setup

Source the Virtualizer tool setup script that ships with your Synopsys installation. Pass the product tier as the last argument so the correct license and tool paths are selected:

Argument	Product
vze	Virtualizer Elite
vzb	Virtualizer Base

```
source /path/to/synopsys/virtualizer/<version>/SLS/linux/setup.sh vze
```

Use **vzb** instead of **vze** if your seat is Virtualizer Base. Omitting the tier may leave the environment incomplete for some workflows; follow your site's Synopsys installation notes.

After sourcing **setup.sh**, the **SNPS_VP_HOME** variable is set and Virtualizer Studio (**vs**) is available on your **PATH**.

Building

1. **GUI:** Open the **.vpproject** file in Virtualizer Studio and build the active configuration.
2. **Headless:** Use the embedded Python API via a build script (e.g. **scripts/build.py**).

Running

Launch the simulation through Virtualizer Studio or directly via the generated launch configuration file:

```
$SNPS_VP_HOME/bin/snps_vp_sh generated/<config>/launch.conf
```

VP configuration files (**.vpcfg**) control runtime parameters such as CPU firmware images, memory maps, clock frequencies, and peripheral settings.

Key Files in a VDK Project

File / Directory	Purpose
*.vdksys	System descriptor — the single source of truth for the IP hierarchy, connections, and library paths (the libraries array acts as the library manager for all dependent/external IP libraries)

File / Directory	Purpose
*.vpproject	Virtualizer Studio project pointer (active config, build type, sim working directory)
vpconfigs/	Simulation scenario configurations (.vpcfg files)
generated/	Build-time artefacts (launch config, properties, dynamic load map)
bin/export/	Exported simulation binary and runtime config

For more detail on tool usage—Virtualizer Studio, command-line utilities, licensing, packaging, and methodology—refer to the Synopsys Virtualizer documentation included with your installation (for example under `$SNPS_VP_HOME/../../Documentation/` after you source `setup.sh`).

2.2 Accellera SystemC (Open-Source)

Accellera SystemC is the open-source reference implementation of the IEEE 1666 SystemC standard. It provides the simulation kernel, TLM-2.0 library, and all the infrastructure needed to compile and run SystemC models without any commercial tooling.

Prerequisites

Requirement	Details
Accellera SystemC	2.3.3 or later — accellera.org/downloads
CMake	3.10 or later
C++ compiler	C++20 capable (GCC 10+, Clang 10+)
Tested host OS	Ubuntu 22.04 LTS (x86_64) and Red Hat Enterprise Linux 8.10 (Ootpa, kernel 4.18.0-553.el8_10.x86_64)

Environment Setup

The CMake build system uses `SYSTEMC_HOME` to locate `systemc.h` and `libsystemc`.

```
export SYSTEMC_HOME=/path/to/systemc-2.3.3
```

Building and Running

A typical project follows the standard CMake workflow:

```
cd <project_directory>
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make
./<output_binary>
```

Common CMake options vary by project but may include build type (Debug, Release, Coverage), feature toggles, and test-vector capture flags.

Key Files in an Accellera Project

File / Directory	Purpose
CMakeLists.txt	Build system entry point
SystemC/include/	IP model headers (module declarations, TLM socket ports)
SystemC/src/	IP model implementation
Tests/	Testbench and test configuration
scripts/	Build helpers and CI/regression scripts

3. Interface Standards

Every IP that plugs into the platform must conform to these standards. This ensures that any IP can be connected to any other IP or bus model in either simulation flow without adapter logic.

Standard	What It Covers	Specification
IEEE 1666 SystemC	Simulation kernel, <code>sc_module</code> hierarchy, <code>sc_signal</code> , processes (<code>SC_METHOD</code> , <code>SC_THREAD</code>)	IEEE Std 1666-2023
TLM-2.0	Transaction-level modelling for memory-mapped communication between IPs	Part of IEEE 1666
<code>tlm_generic_payload</code>	The standard transaction object — carries address, command (read/write), data pointer, byte enables, and response status	TLM-2.0 base protocol
<code>tlm_extension<T></code>	Optional mechanism to attach protocol-specific sideband data to a transaction (e.g. device ID, address type, QoS)	TLM-2.0 extension API

Socket Types

TLM-2.0 defines initiator and target socket roles. Every memory-mapped connection pairs one initiator with one target. The usual helpers in application code are the `simple_*` sockets from `tlm_utils`:

Target (slave) — receives transactions (register/CSR access from a CPU or another initiator).

```
tlm_utils::simple_target_socket<T, 64>
```

Initiator (master) — sends transactions (DMA, memory traffic, upstream requests).

```
tlm_utils::simple_initiator_socket<T, 64>
```

The 64 template parameter is the bus width in bits. Both sides of a connection must use the same width.

The IEEE 1666 SystemC standard (TLM-2.0) also defines other socket classes for bridges, multi-port routing, and direct use of the base protocol API. Examples include:

- `tlm::tlm_initiator_socket` / `tlm::tlm_target_socket` — base-protocol sockets; often wrapped or subclassed by `simple_*` and vendor-specific types.
- `tlm_utils::passthrough_initiator_socket` / `passthrough_target_socket` — forward transactions through a module (e.g. simple bus bridges).
- `tlm_utils::multi_passthrough_initiator_socket` / `multi_passthrough_target_socket` — one logical port fan-out or fan-in to multiple peers.

See the TLM-2.0 chapter in the IEEE 1666 Language Reference Manual for the full socket hierarchy and semantics.

Sideband Signals

In addition to TLM sockets, IPs use standard SystemC signals for clock, reset, and interrupts:

Signal	Direction	Type	Description
Clock	Input	<code>sc_in<bool></code>	System clock (active-edge driven)
Reset	Input	<code>sc_in<bool></code>	Synchronous or asynchronous reset
Interrupt	Output	<code>sc_out<bool></code>	Active-high interrupt to the interrupt controller

Transport Interface

All target sockets must implement the blocking transport callback:

```
void b_transport(tlm::tlm_generic_payload &trans, sc_core::sc_time &delay);
```

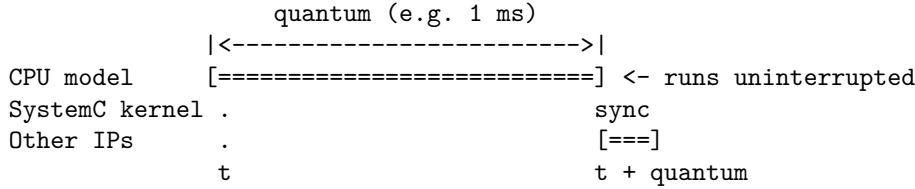
Inside this callback your IP:

1. Reads `trans.get_address()` to determine which register is accessed.
2. Checks `trans.get_command()` (`TLM_READ_COMMAND` or `TLM_WRITE_COMMAND`).
3. Transfers data via `trans.get_data_ptr()`.
4. Sets `trans.set_response_status(tlm::TLM_OK_RESPONSE)` (or an appropriate error code) before returning.

4. Temporal Decoupling and the Quantum

4.1 What Is Temporal Decoupling?

In a cycle-accurate simulation every IP model would synchronize with the SystemC simulation kernel on every clock edge. This is accurate but extremely slow. Temporal decoupling is a TLM-2.0 technique that allows a fast model (typically a CPU instruction-set simulator) to run ahead of the global SystemC simulation time by a bounded amount, called the **quantum**. The model only yields back to the SystemC scheduler when it has consumed a full quantum's worth of local time.



The larger the quantum, the longer the CPU runs without yielding — fewer context switches, higher simulation speed. The trade-off is reduced timing accuracy: peripheral interactions (interrupts, DMA completions, MMIO side-effects) are only observed at quantum boundaries.

4.2 The Quantum Keeper

The quantum keeper (`tlm_utils::tlm_quantumkeeper` in the TLM-2.0 standard) is the mechanism each temporally-decoupled model uses to track how far it has run ahead of SystemC time. Its key operations are:

Operation	What It Does
<code>set_global_quantum(sc_time)</code>	Sets the maximum run-ahead for all models in the simulation (called once at startup)
<code>inc(sc_time)</code>	Adds local elapsed time (e.g. one instruction's execution time)
<code>need_sync()</code>	Returns true when the accumulated local time has reached the quantum — the model must call <code>sc_core::wait()</code> to re-synchronize
<code>sync()</code>	Forces an immediate synchronization with the SystemC kernel
<code>get_local_time()</code>	Returns how far ahead the model currently is

A typical CPU model loop looks like:

```
while (running) {
    execute_one_instruction();
    qk.inc(instruction_time);
    if (qk.need_sync())
        qk.sync();           // yields to SystemC kernel
}
```

4.3 Setting the Quantum in Virtualizer (VDK)

Synopsys Virtualizer provides a built-in model called `quantum_initializer` (`Synopsys/VDKC_MODELS/quantum_initializer`) that calls `tlm_quantumkeeper::set_global_quantum()` during elaboration. It is typically instantiated inside an infrastructure group in the `.vdksys` system descriptor:

```
{
    "name" : "quantum_initializer",
    "kind" : "model_instance",
    "model" : {
        "id" : "Synopsys/VDKC_MODELS/quantum_initializer",
        "version" : "latest"
    },
    "parameter_sets" : [ {
        "name" : "scml_properties",
        "overrides" : [ {
```

```

        "model_parameter_name" : "/quantum_value",
        "value" : "50"
    }, {
        "model_parameter_name" : "/quantum_unit",
        "value" : "SC_US"
    } ]
} ]
}

```

Parameter	Description	Typical Values
quantum_value	Numeric value of the quantum	50, 1000, 1000000
quantum_unit	SystemC time unit	SC_PS, SC_NS, SC_US, SC_MS, SC_SEC

The effective quantum is $\text{quantum_value} \times \text{quantum_unit}$. For example, $\text{quantum_value} = 1000$ with $\text{quantum_unit} = \text{SC_US}$ gives a 1 ms quantum.

Overriding at runtime: You do not need to edit the `.vdksys` to change the quantum. Override it in the VP configuration file (`.vpcfg`) using an SCML property override:

```

<paramOverrides
  key="Platform.Subsystem.Infrastructure.quantum_initializer.#SCML_PROPERTIES#quantum_value"
  value="1000000"/>

```

The key is the hierarchical path to the `quantum_initializer` instance followed by `#SCML_PROPERTIES#quantum_value`. This lets you tune the quantum per simulation scenario without modifying the platform source.

4.4 Setting the Quantum in Accellera SystemC

In a pure Accellera SystemC environment there is no `quantum_initializer` model. Instead, set the global quantum directly in `sc_main()` before calling `sc_start()`:

```

#include <tlm_utils/tlm_quantumkeeper.h>

int sc_main(int argc, char *argv[]) {
    // Set global quantum to 1 ms
    tlm::tlm_global_quantum::instance().set(sc_core::sc_time(1, sc_core::SC_MS));

    // ... instantiate platform, bind sockets ...

    sc_core::sc_start();
    return 0;
}

```

Each CPU or initiator model that supports temporal decoupling should create its own `tlm_quantumkeeper` instance, which automatically reads the global quantum value.

4.5 Best Practices for Linux Boot

Booting a Linux kernel on a virtual platform is an MMIO-intensive workload. The kernel probes peripherals, programs page tables, initializes drivers, and writes thousands of characters to the console — each of these is a TLM transaction that may force a quantum synchronization. Choosing the right quantum value has a significant impact on boot time.

Reference build. A RISC-V based reference platform boots **Linux 6.6.132** (6.6.y LTS), built with the `riscv64-unknown-linux-gnu- GCC 15.2.0` toolchain pinned in §2.1, with OpenSBI as M-mode firmware (`fw_payload.elf`). The quantum guidance below was tuned against that kernel.

Scenario	Recommended Quantum	Rationale
Bare-metal firmware / RTOS	10 – 100 μ s	Low latency needed for tight polling loops and real-time deadlines
Linux boot (normal)	500 μ s – 1 ms	Good balance between speed and peripheral responsiveness
Linux boot (speed-optimized)	1 – 10 ms	Maximizes CPU throughput; acceptable for non-interactive boot
Interactive Linux session	100 – 500 μ s	Lower quantum improves responsiveness of shell, networking, etc.

General guidelines:

- **Start large, reduce if needed.** Begin with a quantum of 1 ms for Linux boot. If peripherals misbehave (missed interrupts, DMA timeouts), reduce the quantum until the issue resolves.
- **Larger quantum = faster boot.** Every synchronization point has overhead. A 1 ms quantum may execute thousands of instructions between syncs, while a 1 μ s quantum syncs after just a few — the difference in wall-clock boot time can be 10 $\times$ or more.
- **Peripheral-heavy phases are sensitive.** During PCIe enumeration, DMA setup, or interrupt-driven I/O, a very large quantum can cause the CPU to run past the point where it should have observed a completion or interrupt. If you see hangs during device probing, try reducing the quantum.
- **Console output is a quantum bottleneck.** Each UART register write is a TLM transaction. Suppressing verbose kernel output (quiet `loglevel=1` in `bootargs`) or using interrupt-driven UART with FIFO batching dramatically reduces the number of synchronization points.
- **Use different quanta for different phases.** Some platforms support changing the quantum at runtime (e.g. large quantum during boot, smaller quantum once userspace starts). Check whether your CPU model supports dynamic quantum adjustment.

4.6 Memory and Transaction Ordering in Loosely-Timed Simulation

TLM-2.0 Loosely-Timed (LT) simulation uses `b_transport()` — a blocking, call-and-return function — as its primary transport mechanism. This has direct implications for how memory and bus ordering rules (e.g. PCIe posted/non-posted ordering, AMBA AXI ordering) are handled.

How ordering works in LT mode. Within a single CPU thread, program order is preserved. Each `b_transport()` call from the CPU model is a synchronous function call: the initiator (CPU) blocks until the target returns. The next instruction only executes after the previous transaction completes. This means that within a single core, loads and stores execute in strict program order — there is no out-of-order pipeline or reordering buffer at the TLM level.

Across multiple cores, ordering is weakly defined. Each core runs in its own SystemC thread and may be at a different point in local quantum time. Transactions from different cores can interleave at the SystemC scheduler in any order relative to global simulation time. There is no built-in memory barrier or fence mechanism in TLM-2.0 LT. If your software relies on cross-core ordering (e.g. spin-locks, shared-memory synchronization), the quantum must be small enough that synchronization points occur frequently, or the target must explicitly call `wait(local_quantum_time)` to force time synchronization before processing a transaction.

Protocol-specific ordering rules are relaxed. Real hardware buses enforce strict ordering semantics:

- PCIe requires posted writes to complete before non-posted requests in the same direction, and completions to be ordered relative to their requests.
- AMBA AXI has ordering rules based on transaction IDs and memory attributes.

In LT simulation, these protocol-level ordering rules are not enforced by the TLM-2.0 base protocol. Because `b_transport()` is an atomic call-and-return, there is no pipeline and therefore no opportunity for transactions to be reordered within a single initiator's thread. The simulation effectively operates under a sequentially-consistent model for single-threaded initiators and a weakly-ordered model across multiple initiators.

When does this matter? For most software development and IP validation use-cases, the LT ordering model is sufficient — software running on a single core sees all its transactions complete in program order, and most drivers do not depend on cross-core transaction ordering at the bus level.

Ordering becomes relevant when:

- **Multi-core software** uses relaxed memory ordering and depends on hardware barriers (`fence`, `dmb`, `dsb`) to enforce visibility. In LT simulation, these barriers typically force a quantum sync (`qk.sync()`) rather than modelling bus-level ordering.
- **DMA engines** that must observe ordering between descriptor fetches and data writes. If modelled at LT level, ordering is guaranteed by sequential `b_transport()` calls within the DMA engine's SystemC thread.
- **PCIe completion ordering** in multi-function endpoints. In LT mode, completions are returned inline with the `b_transport()` call, so ordering is implicit.

DMI bypasses all ordering. When Direct Memory Interface (DMI) is active, the CPU reads and writes directly to the host memory backing a SystemC memory model — bypassing `b_transport()` entirely. DMI accesses have no ordering guarantees relative to non-DMI transactions, because no SystemC process or target callback is invoked. This is the expected trade-off: DMI delivers maximum simulation speed (no TLM overhead per access) at the cost of not modelling bus-level side-effects or ordering.

If your IP depends on observing every memory access (e.g. an IOMMU or a bus monitor), it must invalidate the DMI region for the address ranges it needs to observe, forcing the CPU back to `b_transport()`. This is done by calling `invalidate_direct_mem_ptr()` on the target socket.

Memory consistency. Memory consistency defines when a write performed by one core becomes visible to other cores. In real hardware this is governed by the cache coherence protocol (MESI, MOESI, etc.) and the architecture's memory model (e.g. RISC-V RVWMO, Arm weakly-ordered, x86 TSO). TLM-2.0 LT simulation does not model caches or coherence protocols, which has several practical consequences:

Writes are instantly visible. When a CPU core completes a `b_transport()` write, the data is stored directly in the target's backing memory (a host-side buffer). Any subsequent `b_transport()` read from any other core to the same address will see the updated value immediately. There is no write buffer, no store queue, and no cache that could hold stale data. In this sense, LT simulation provides strong consistency for the data itself — stronger than most real hardware.

Timing of visibility is inaccurate. Although the data is instantly visible, the simulated time at which the visibility occurs may be wrong. If Core A writes at local time $T=100\mu\text{s}$ (but global SystemC time is only $T=50\mu\text{s}$ because Core A is running ahead), Core B at local time $T=70\mu\text{s}$ can already see Core A's write — even though in real hardware the write has not happened yet from Core B's perspective. This is an inherent artifact of temporal decoupling: all cores share the same host memory, but their local clocks are not synchronized.

No cache coherence modelling. TLM-2.0 LT does not model cache lines, invalidation, snooping, or coherence states. Implications:

- **No false sharing.** In real hardware, two cores writing to different words in the same cache line cause coherence traffic and performance degradation. In LT simulation this effect is invisible.
- **No coherence fences.** Hardware fence instructions (`fence`, `dmb`, `dsb`) are typically mapped to a quantum synchronization (`qk.sync()`) in the CPU model, which forces a yield to the SystemC kernel. This ensures other cores' pending events are processed, approximating the effect of a barrier, but does not model actual coherence protocol traffic.
- **No MESI/MOESI state transitions.** Software that explicitly manages cache state (e.g. DMA buffer cache maintenance, self-modifying code with `fence.i`) will execute correctly in LT simulation because there are no caches to be stale, but the performance characteristics and timing will differ from real hardware.

DMI and multi-core consistency. When multiple cores have DMI pointers to the same memory region, reads and writes go directly to host memory with no SystemC involvement. Visibility between cores depends entirely on the host machine's native memory model (typically x86 TSO or Arm weakly-ordered). On an x86 host, this effectively gives sequential consistency between DMI-based accesses. On an Arm host, explicit host-side barriers may be needed in the simulator implementation — but this is handled by the SystemC kernel and CPU model implementors, not by the IP developer.

Practical guidance:

Real hardware concept	LT simulation equivalent
Write buffer / store queue	Not modelled — writes commit immediately
L1/L2 cache	Not modelled — all accesses go to backing memory
Cache coherence (MESI)	Not modelled — data is always coherent
Memory barriers (<code>fence</code>)	Mapped to quantum sync (<code>qk.sync()</code>)

Real hardware concept	LT simulation equivalent
Cache maintenance (dcache flush)	No-op in simulation (no caches to flush)
Acquire/release semantics	Approximated by quantum sync timing

For most software development and validation, this simplified consistency model is an advantage: software bugs caused by missing barriers or incorrect cache maintenance are unlikely to manifest in simulation (the data is always coherent). Conversely, if you need to validate barrier correctness or coherence-dependent algorithms, an LT virtual platform is not the appropriate tool — a cycle-accurate or AT (Approximately-Timed) model with explicit cache and coherence modelling would be needed.

5. How to Integrate a New IP

5.1 Common Requirements

Regardless of which simulation flow you target, your IP must:

1. **Be a SystemC module** — inherit from `sc_core::sc_module`.
2. **Expose TLM-2.0 sockets** — at least one `simple_target_socket` for register access; optionally a `simple_initiator_socket` if the IP initiates transactions (DMA, memory fetches, etc.).
3. **Register a `b_transport()` callback** on every target socket.
4. **Provide clock, reset, and interrupt ports** as `sc_in<bool>` / `sc_out<bool>`.
5. **Use `tlm_generic_payload`** for all memory-mapped data transfer.

Here is a minimal IP skeleton that satisfies these requirements:

```
#include <systemc>
#include <tlm.h>
#include <tlm_utils/simple_target_socket.h>
#include <tlm_utils/simple_initiator_socket.h>

class MyIP : public sc_core::sc_module {
public:
    tlm_utils::simple_target_socket<MyIP, 64>    p_reg_socket;
    tlm_utils::simple_initiator_socket<MyIP, 64> p_dma_socket;

    sc_core::sc_in<bool> p_clk;
    sc_core::sc_in<bool> p_reset;
    sc_core::sc_out<bool> p_int;

    SC_CTOR(MyIP)
        : p_reg_socket("p_reg_socket")
        , p_dma_socket("p_dma_socket")
    {
        p_reg_socket.register_b_transport(<this>, &MyIP::reg_b_transport);
    }

private:
    void reg_b_transport(tlm::tlm_generic_payload &trans,
                        sc_core::sc_time &delay)
    {
        uint64_t    addr = trans.get_address();
        unsigned char *data = trans.get_data_ptr();
        unsigned int  len  = trans.get_data_length();

        if (trans.get_command() == tlm::TLM_READ_COMMAND) {
            // Populate data buffer with register value at addr
        } else {
```

```

        // Write data buffer contents into register at addr
    }

    trans.set_response_status(tlm::TLM_OK_RESPONSE);
}
};

```

5.2 VDK Flow — Declarative Integration

In the Synopsys Virtualizer flow, IPs are integrated by editing the system descriptor JSON file (`.vdksys`). There is no hand-written C++ binding code.

Step 1 — Package your IP as a shared library. Build your SystemC model into a `.so` using Synopsys model packaging conventions. The library must export a factory that Virtualizer can use to instantiate the `sc_module`.

Step 2 — Register the library path. Add your library directory to the `libraries` array in the `.vdksys`, or reference it via an environment variable set in the project's setup script:

```

"libraries" : [
    "${SNPS_VP_HOME}/virtualizerstudio/lib",
    "${MY_IP_LIB_DIR}",
    ...
]

```

Step 3 — Declare the IP instance. Add an entry to the `instances` array inside the appropriate hierarchy group in the `.vdksys`:

```

{
    "name" : "MY_IP",
    "kind" : "model_instance",
    "model" : {
        "id" : "Vendor/Library/ModelName",
        "version" : "latest"
    },
    "interfaces" : [
        {
            "name" : "reg_target",
            "kind" : "slave",
            "protocol_id" : "tlm2_gp"
        },
        {
            "name" : "dma_initiator",
            "kind" : "master",
            "protocol_id" : "tlm2_gp"
        }
    ]
}

```

- `name` — must match the TLM socket name on the SystemC module.
- `kind` — `"slave"` for target sockets, `"master"` for initiator sockets.
- `protocol_id` — use `"tlm2_gp"` for standard TLM-2.0 generic payload.

Step 4 — Wire the connections. Add an entry to the `connections` array to bind your IP to the platform's address decoder (shared memory map) at a specific address range:

```

{
    "endpoints" : [
        {
            "instance" : ["GroupName", "SharedMemoryMap"],
            "interface" : "intf",
            "side" : "internal"
        }
    ]
}

```

```

    },
    {
        "instance" : ["GroupName", "MY_IP"],
        "interface" : "reg_target",
        "side"      : "external"
    }
],
"decoded_parameters" : {
    "start" : "0x48000000",
    "size"  : "0x00010000",
    "decoded" : true
}
}
}

```

Step 5 — Connect clock and reset. Wire clock and reset ports using protocol-typed connections ("protocol_id": "CLOCK" / "RESET"), linking your IP to the platform's clock and reset generators.

Step 6 — Configure parameters (optional). Override model parameters at runtime through the VP configuration file (.vpcfg) using SCML property overrides, or interactively through Virtualizer Studio.

You do not have to edit the .vdksys by hand in a text editor. Virtualizer Studio can update the same system descriptor through its graphical flow: drag IP components from the library into the Design Hierarchy view, place instances, and connect interfaces there; the tool persists those changes back into the .vdksys. Use whichever mix of GUI editing and direct JSON edits fits your workflow.

5.3 Accellera Flow — Programmatic Integration

In the open-source flow, you write standard C++ to instantiate your IP and bind its sockets.

Step 1 — Implement the IP. Write your `sc_module` as shown in Section 5.1.

Step 2 — Instantiate and bind in the top module. Create a top-level `sc_module` (or use `sc_main`) to instantiate your IP alongside the testbench memory and bind sockets:

```

class PlatformTop : public sc_core::sc_module {
public:
    std::shared_ptr<MyIP> ip;
    std::shared_ptr<Memory> mem;

    sc_signal<bool> clk_sig, rst_sig, int_sig;

    PlatformTop(sc_core::sc_module_name name)
        : sc_core::sc_module(name)
    {
        ip = std::make_shared<MyIP>("my_ip");
        mem = std::make_shared<Memory>("memory");

        // Bind TLM-2.0 sockets
        mem->initiator_socket(ip->p_reg_socket); // testbench -> IP registers
        ip->p_dma_socket(mem->target_socket);     // IP DMA -> memory

        // Bind sideband signals
        ip->p_clk(clk_sig);
        ip->p_reset(rst_sig);
        ip->p_int(int_sig);
    }
};

```

Step 3 — Add sources to CMake.

```

add_executable(sim
    src/MyIP.cpp
    tb/PlatformTop.cpp
    tb/main.cpp
)

target_include_directories(sim PRIVATE
    ${SYSTEMC_INCLUDE_DIR}
    include/
)

target_link_libraries(sim ${SYSTEMC_LIB})

```

Step 4 — Use TLM extensions for sideband data (optional). If your IP needs to carry protocol-specific metadata alongside the standard payload (e.g. PCIe device ID, IOVA address type, QoS fields), define a `tlm_extension`:

```

class MySideband : public tlm::tlm_extension<MySideband> {
public:
    uint32_t device_id;
    uint8_t  addr_type;

    tlm_extension_base* clone() const override {
        auto *ext = new MySideband();
        ext->device_id = device_id;
        ext->addr_type = addr_type;
        return ext;
    }

    void copy_from(const tlm_extension_base &other) override {
        auto &o = static_cast<const MySideband &>(other);
        device_id = o.device_id;
        addr_type = o.addr_type;
    }
};

```

Attach it to outgoing transactions:

```

tlm::tlm_generic_payload trans;
auto *ext = new MySideband();
ext->device_id = 0x1234;
trans.set_extension(ext);

```

6. TLM-2.0 Quick Reference

Concept	VDK (.vdksys JSON)	Accellera (C++)
Target (slave)	"kind": "slave", "protocol_id": "tlm2_gp"	<code>tlm_utils::simple_target_socket<T, 64></code>
Initiator (master)	"kind": "master", "protocol_id": "tlm2_gp"	<code>tlm_utils::simple_initiator_socket<T, 64></code>
Address decode	"decoded_parameters" in connections	Manual routing or decoder module
Blocking transport	Handled by Virtualizer runtime	Register <code>b_transport()</code> on target socket
Clock / Reset	"protocol_id": "CLOCK" / "RESET" connections	<code>sc_in<bool></code> bound to <code>sc_signal<bool></code>
Interrupt	Signal connection in .vdksys	<code>sc_out<bool></code> bound to <code>sc_signal<bool></code>

7. Integration Checklist

Use this checklist when adding any new IP to the platform, regardless of simulation flow:

- ☐ IP inherits from `sc_core::sc_module`.
- ☐ At least one `simple_target_socket<T, 64>` is exposed for register access.
- ☐ `b_transport()` is registered on every target socket and handles `TLM_READ_COMMAND` / `TLM_WRITE_COMMAND`.
- ☐ `trans.set_response_status()` is called before returning from `b_transport()`.
- ☐ `sc_in<bool>` ports are provided for clock and reset.
- ☐ `sc_out<bool>` port is provided for interrupt (if applicable).
- ☐ A base address and address range are defined for the IP's register window.
- ☐ IP is instantiated and sockets are bound in the platform top (Accellera) or declared in the `.vdksys` (VDK).
- ☐ (Optional) A `tlm_extension<T>` is defined if the IP requires protocol-specific sideband data.
- ☐ (Optional) `simple_initiator_socket<T, 64>` is exposed if the IP initiates transactions (DMA, memory access).